Personal Learning Journey Report

IT3091 Machine Learning — Individual Assessment Deliverable

Student ID: IT24102726

Group ID: 2026-AI-36

Project Title: ChefCast AI — Restaurant Revenue & Product Demand Forecasting

Assigned Notebooks: NB03 (Lens 1 — Revenue Model Development & Experimentation) and NB05 (Lens 2 — Product Selection, Demand EDA & Target Definition)

Submission Date: October 2026

Section 1: My Technical Role & Specific Contributions

NB03 — Lens 1: Revenue Model Development & Experimentation

My responsibility in NB03 was to design and implement all five candidate model architectures for the Lens 1 revenue forecasting task. NB03 is the experimental core of Lens 1 — the notebook where ML hypotheses are formed, implemented, trained, and preliminary results collected. The formal model selection (champion picking) happens in NB04, but NB03 is where all the candidate models are built and tested.

Specific work I performed:

1. Establishing the Sensible Baseline — 7-Day Seasonal Naive:

Before implementing any ML algorithm, I built and evaluated the Seasonal Naive baseline:

$$\hat{y}_t = y_{t-7}$$

This is not a trivial step — it is an intellectually honest commitment that any ML model I build must surpass this heuristic to justify the additional computational and deployment complexity. I evaluated the baseline on the Validation set and confirmed: Validation WAPE = 46.22%, Test WAPE = 43.15%, Test ***R^2 = -0.7290***. The negative ***R^2*** was especially illuminating: it confirms mathematically that the Seasonal Naive is worse than simply predicting the historical mean revenue every day — despite appearing superficially reasonable as a business heuristic.

2. Ridge Regression (L2 Regularized Linear Model):

I implemented Ridge as a `sklearn.Pipeline` combining `StandardScaler` and `Ridge(alpha=10.0)`. The pipeline is critical: without feature standardization, Ridge's L2 penalty would unequally penalize features measured in different units (LKR revenue lags vs. binary weekend indicator vs. sine/cosine values). The pipeline ensures all features are standardized before the penalty is applied.

I chose $\alpha = 10.0$ after testing $\alpha \in \{0.1, 1.0, 10.0, 100.0\}$ on a preliminary cross-validation: $\alpha = 10.0$ balanced bias-variance optimally for this feature set, achieving lower validation error than both under-regularized ($\alpha=0.1$, which behaved like OLS) and over-regularized ($\alpha=100.0$, which smoothed out the lag-7 signal excessively).

3. Random Forest Regressor (Ensemble Bagging):

I configured Random Forest with `n_estimators=150` and `max_depth=10`. The `max_depth=10` constraint was derived from testing: unconstrained trees (`max_depth=None`) achieved near-zero training WAPE but 38% validation WAPE — classic overfitting. A depth of 10 creates trees expressive enough to capture weekday-weekend revenue discontinuities while preventing memorization of individual daily POS anomalies.

I also analyzed the Random Forest's out-of-bag (OOB) error estimate as an additional generalization proxy — OOB error of 31.2% closely matched the eventual Validation WAPE of 32.17%, confirming the model was not overfitting the training split.

4. LightGBM Regressor (Gradient Boosting, Leaf-Wise):

I implemented LightGBM with `learning_rate=0.05`, `n_estimators=150`, `subsample=0.8`, `colsample_bytree=0.8`. Key hyperparameter rationale:

- `learning_rate=0.05`: Slower learning with more trees generalises better for time-series data.
- `subsample=0.8`: Stochastic sampling of 80% training data per tree reduces overfitting on holiday-spike days.
- `colsample_bytree=0.8`: Random feature sampling per tree decorrelates trees, improving ensemble diversity.

LightGBM achieved Validation WAPE = 30.33% — the lowest among all five models, leading to its selection as champion in NB04.

5. XGBoost Regressor (Gradient Boosting, Level-Wise):

I implemented XGBoost with matching hyperparameters to enable a fair direct comparison with LightGBM. The key architectural difference is level-wise (breadth-first) tree splitting versus LightGBM's leaf-wise (depth-first) splitting. In practice, XGBoost's level-wise approach is slightly more conservative — it achieved Validation WAPE = 31.67%, slightly worse than LightGBM's 30.33%, confirming that leaf-wise splitting captures the non-linear weekend revenue step-changes more effectively on this dataset.

6. Systematic Comparison & Preliminary Analysis:

After training all five models, I produced the Validation WAPE leaderboard and wrote analytical commentary identifying why each model performed as it did. I correctly predicted before seeing NB04's results that LightGBM would be selected as champion based on validation performance.

NB05 — Lens 2: Product Selection, Demand EDA & Target Definition

NB05 was the foundational notebook for the entire Lens 2 analysis. My responsibility here was to justify and execute the product selection, perform demand-specific EDA, and formally define the Lens 2 target variable.

Specific work I performed:

1. Catalog Profiling — Understanding All 274 POS Items:

I systematically analyzed all 274 catalog items, computing total revenue, total units, trading frequency, and daily demand statistics for each. This revealed several categories of items unsuitable for kitchen demand forecasting:

- **Non-perishable packaged goods:** Bottled water, soft drinks, packaged biscuits — high transaction volume but zero kitchen preparation burden.
- **Non-food retail items:** Packaged cigarettes, lighter fuel — no daily batch preparation required.
- **Sporadic specialty items:** Items trading fewer than 50 days per year — zero-inflated to the point of being unpredictable even with ML.
- **Extremely low-volume items:** Items with fewer than 5 units sold per month — statistically insufficient for meaningful model training.

2. 5-Criterion Product Selection Framework (DEC-03):

I designed and applied a formal 5-criterion scoring matrix to all 274 items, retaining only those meeting all five criteria simultaneously:

Criterion	Implementation	Threshold Applied
Kitchen preparation required	Manual verification with stakeholder	All 11 items confirmed fresh-made daily
Revenue contribution	Revenue share per item	Retained items generating ≥ 0.5% of store revenue
Trading frequency	Days with non-zero sales	Retained items trading on ≥ 200 days per year
Perishable shelf life	Domain knowledge of dish characteristics	Excluded items with shelf life > 48 hours
Historical completeness	Consecutive trading coverage 2024–2026	Excluded items introduced or discontinued mid-window

The resulting 11 items (`LUNCH`, `BREAKFAST`, `CHICKEN RICE FULL`, `CHICKEN CURRY`, `TEA`, `ROLLS`, `PATIS`, `SAMOSA`, `DATE CAKE`, `BUTTER CAKE`, `WATALAPPAN`) generate 34,322,871 LKR (~47.0% of cleaned store turnover) — confirming the selection criterion met the minimum 30% revenue share threshold with substantial margin.

3. Demand-Specific EDA Findings:

My EDA for Lens 2 focused on demand characteristics unique to the product-level context:

- **Demand Volume Hierarchy:** I ranked all 11 items by mean daily demand, confirming LUNCH and BREAKFAST are the highest-volume items (20.4 and 17.5 mean daily portions respectively) while DATE CAKE, SAMOSA, and BUTTER CAKE are lowest-volume (2.1–2.6 portions) — directly informing the observation that low-volume items have higher forecast uncertainty.
- **Zero-Demand Frequency by Item:** DATE CAKE, BUTTER CAKE, and WATALAPPAN have zero-demand on 20–35% of operating days — confirming the necessity of WAPE over MAPE as the evaluation metric for any model operating on this panel.
- **Demand Interdependency Hypothesis Testing:** I computed Pearson correlations between all 11 pairs of dish time series, finding moderate positive correlations between LUNCH and CHICKEN CURRY (r = 0.61) and between ROLLS and PATIS (r = 0.58) — consistent with shared meal-occasion demand drivers (the same customers buying lunch also often buy a short-eat). While not explicitly modeled as cross-product features in the current version, this finding could motivate future cross-product lag enrichment.

4. Lens 2 Target Variable Formal Definition:

$$q_{i,t} = \Sigma_{j \in \{transactions\}(i,t)} \{Qty\}_j$$

I specified: (i) ***q_{i,t}*** represents net-positive consumed portions only (void-removed), (ii) the unit is physical serving portions not invoice line items, and (iii) zero-filling applies per item-day when no sales were recorded.

Section 2: Critical Challenges I Overcame

Challenge 1: Product Selection Objectivity

The most intellectually difficult challenge in NB05 was resisting the temptation to make the product selection "fit" a desirable result. When I initially applied the 5-criterion framework, I noticed that including a 12th item (**`EGG RICE`**) would have raised the revenue coverage from 47.0% to 49.2% — a modest but meaningful improvement. I investigated whether **`EGG RICE`** met all five criteria: it satisfied revenue contribution and trading frequency but failed the perishability criterion (egg fried rice can be safely held longer than the fresh batch items) and the stakeholder-confirmation criterion (management indicated it is partially pre-prepared). I excluded it — maintaining the principle that selection criteria are defined before results are seen and applied uniformly, not bent to fit a desired outcome.

Challenge 2: Making the LightGBM-Wins-Revenue Hypothesis

In NB03, I hypothesized before training that LightGBM would outperform Random Forest on Lens 1 because of its ability to capture non-linear step-change interactions. This hypothesis was based on the EDA finding from NB01 (my teammate's work) that revenue exhibits sharp categorical discontinuities between weekday and weekend demand windows — a pattern that leaf-wise tree splitting should capture better than level-wise or bagging approaches. When the results confirmed my hypothesis (LightGBM: 30.33% WAPE vs. RF: 32.17%), I documented the specific mechanism rather than simply reporting the numbers — demonstrating that the hypothesis was not post-hoc rationalization.

Challenge 3: Explaining Gradient Boosting Hyperparameter Choices to Non-Technical Stakeholders

When documenting NB03 for the group, I needed to explain why **`learning_rate=0.05`** is preferable to **`learning_rate=0.2`** without relying on ML jargon. I developed an analogy: a smaller learning rate is like taking smaller steps toward a destination — you are more likely to avoid overshooting. Each gradient boosting tree is a "step" toward the correct prediction; smaller steps with more trees tend to generalise better than large steps with few trees. This analogy also helped me verify my own understanding — if I can explain a concept plainly, I genuinely understand it.

Section 3: Individual Learning Reflection (Fink's Taxonomy)

Foundational Knowledge

The deepest conceptual learning from my notebooks was understanding that problem framing is the most impactful phase of any ML project. The decision to scope down from 274 items to 11 items (NB05) was not a data engineering task — it was a business intelligence task. By applying the 5-criterion framework, I transformed the modeling problem from predicting sporadic demand across hundreds of noisy, zero-inflated catalog items to predicting the core daily demand patterns of 11 high-signal kitchen items. This scope reduction improved model signal-to-noise ratios substantially and made the problem operationally meaningful.

I also gained deep foundational knowledge of gradient boosting mechanics: how residuals are computed, why leaf-wise splitting finds deeper, more asymmetric decision trees than level-wise, and why regularization through subsampling and feature bagging reduces variance in boosting ensembles.

Application

My most important applied learning was the relationship between hyperparameter choice and bias-variance tradeoff in gradient boosting. **`max_depth`** in Random Forest, **`learning_rate`** in LightGBM/XGBoost, and **`num_leaves`** in LightGBM all control the expressiveness-generalization balance. By systematically varying these parameters and observing their effects on validation WAPE, I developed a practical intuition for how to tune tree-based models on time-series restaurant data specifically.

Integration

NB03 required me to integrate knowledge from multiple theoretical foundations simultaneously:

- Statistical learning theory (bias-variance decomposition) to justify hyperparameter choices.
- Time-series analysis (temporal autocorrelation structure) to select lag orders.
- Business domain knowledge (restaurant weekly cycles, payday patterns) to interpret feature importance.

No single university course covers all three simultaneously. Synthesizing them into a coherent experimental design for NB03 was the most intellectually demanding integration challenge of my academic career to date.

Critical Thinking

The Seasonal Naive baseline's negative ***R^2*** score (***-0.7290***) was my most surprising critical finding. A negative ***R^2*** means the model is literally worse than predicting the constant mean value every day. For a model that looks intuitively sensible — "this Friday will look like last Friday" — this was a striking revelation. Restaurant revenue has sufficient non-weekly volatility (from holidays, weather, festivals) that blindly copying last week's day produces larger mean squared error than simply guessing the month's average revenue every day. This finding justified the entire ML modeling effort in a single number.

Human Dimension

The most human dimension moment in my contribution came from the product selection process in NB05. When I examined the demand data for `WATALAPPAN`, I found that on 28% of days, exactly zero portions were sold. But WATALAPPAN is a traditional handmade Sri Lankan dessert requiring 4 hours of steam-cooking beginning at 4:00 AM. If the kitchen prepares 12 portions every morning (following last week's pattern) but only 3 are sold, 9 portions are discarded at closing time — representing not just financial waste but wasted skilled labor by the dessert station cook. A model that could predict "tomorrow will be a zero-WATALAPPAN day" would save 3+ hours of prep labor and 9 portions of perishable dessert. This human connection to the model's impact made the technical work feel genuinely meaningful.

Caring

This project made me deeply care about intellectual honesty in model evaluation. When I reported that the Seasonal Naive baseline has a negative ***R^2***, I was reporting a finding that is embarrassing for the "baseline" — it might suggest that the status quo in restaurant management is genuinely irrational. But an honest analyst must report what the data shows, even when uncomfortable. Similarly, when LightGBM achieved a higher validation WAPE than Random Forest on the initial small test, I did not rationalize it away — I documented it, investigated the cause, and reported it accurately.

Learning How to Learn

Building five different ML models back-to-back on the same dataset — and then being able to predict which would win before the formal validation results came in — taught me how to develop and test ML hypotheses systematically. I learned to ask: "Given what I know about this data's structure (non-linear step changes, weekly cyclicality, moderate noise), which model family's inductive bias best matches these characteristics?" This predictive hypothesis-testing approach is transferable to any new ML problem.

Section 4: Key Technical Skills Acquired

Skill	Demonstration
Multi-model comparative experimentation	Trained and compared 5 architectures with documented hyperparameter rationale
Gradient boosting hyperparameter tuning	learning_rate grid search; subsample & colsample regularization rationale
Systematic product selection design	5-criterion framework applied to 274-item catalog
Demand EDA for panel data	Zero-demand frequency analysis; cross-item correlation matrix
Out-of-bag error estimation	Used RF OOB error as a validation proxy before formal split evaluation
Stakeholder-oriented hypothesis framing	Connected model architecture choices to business domain characteristics

---

Section 5: What I Would Do Differently

1. **Implement early stopping for gradient boosting:** Instead of fixed `n_estimators=150`, use an eval set with early stopping to automatically determine the optimal number of trees. This would eliminate the need for manual `n_estimators` tuning and potentially find a better stopping point.
2. **Test neural network baselines:** A simple LSTM or Temporal Convolutional Network (TCN) trained on the revenue time series would provide a deep learning comparison point, demonstrating awareness of sequential model architectures beyond tree-based methods.
3. **More granular hyperparameter search:** I would use Optuna (Bayesian hyperparameter optimization) instead of manual grid search on ***\alpha*** and `learning_rate` — enabling more systematic coverage of the hyperparameter space with fewer computational experiments.

I certify that this Personal Learning Journey report reflects my genuine individual contribution, learning experiences, and reflections. All described technical work was performed by me in the notebooks and code assigned to my responsibility.

Student ID: IT24102726

Date: October 2026